VEXON DEVELOPER HANDBOOK · GUIDE 08

Deployment and Production Maintenance

How VEXON runs locally and on Render today, what production needs, and a careful plan for a future VPS

Written for	A student who will keep VEXON running for real users and may move it to a college-owned server
Project	VEXON / CampusConnect (repository: this folder's parent project)
Evidence	Generated from a direct inspection of the repository source code
Repository state	branch fix/csrf-legacy-cookie, commit af623fb, plus 126 uncommitted working-tree change(s)
Generated	16 October 2026, 21:11
Rebuild	python docs/developer-handbook/build_handbook.py

Accuracy rule: every statement about VEXON in this guide is based on files in the repository. Anything that could not be confirmed is labelled 'unverified', 'planned' or 'hypothetical'. No secrets, passwords or real student data are reproduced.

Contents

Scope and honesty . . . 3
Local development setup (verified) . . . 3
The current hosting setup: Render (configured in render.yaml) . . . 3
Sleeping and waking (free plan) . . . 4
Required environment variables (names only) . . . 4
Build and start commands . . . 5
HTTPS, proxies and secure networking . . . 5
Process management, logs and health checks . . . 6
Backups and tested restoration . . . 6
Monitoring, incidents and rollback . . . 6
Security and authorization checks before going live . . . 7
Planning a move to a VPS . . . 7
Local build vs production-ready system . . . 8
Source references . . . 8

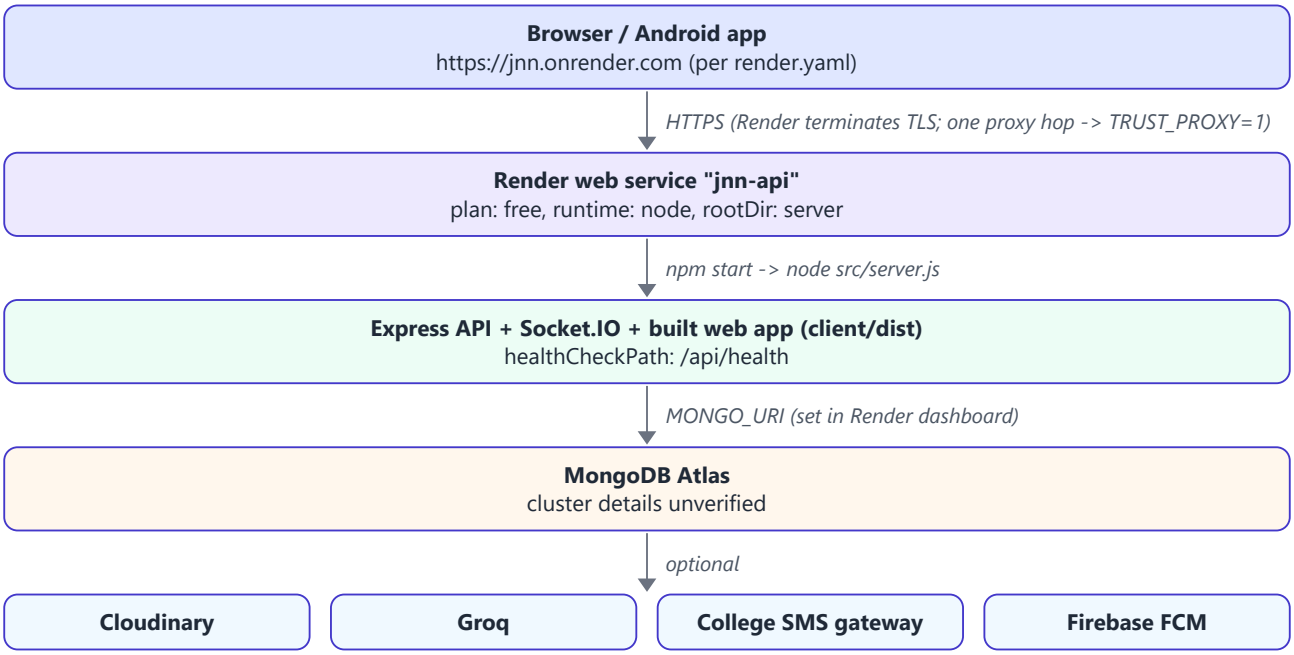
Scope and honesty

> **Evidence status**
> This guide describes the deployment configuration **found in the repository** (`render.yaml`, `.github/workflows/`, `server/src/app.js`, `server/src/utils/keepAlive.js`). It could not see the Render dashboard, the Atlas project, DNS or any server. Statements about those are marked **unverified**. Nothing was deployed during this audit.

Local development setup (verified)

Piece	How it runs locally
MongoDB	Windows service "MongoDB Server" on `127.0.0.1:27017`. The dev database is `campusconnect`; tests use `cmp_test`
API	`npm run dev --prefix server` (nodemon on `src/server.js`, port 5000)
Web	`npm run dev --prefix client` (Vite on port 5173; proxies `/api`, `/uploads`, `/socket.io` to 5000)
Both together	`npm run dev` at the root (`concurrently`)
Kotlin app	Emulator with `-Pvexon.apiUrl=http://10.0.2.2:5000`, or a phone on the same Wi-Fi pointed at the PC's LAN IP
Config	`server/.env` (copy of `server/.env.example`; never committed)

The current hosting setup: Render (configured in `render.yaml`)



What `render.yaml` defines:

- **Build:** `npm install && cd ../client && npm install --include=dev && npm run build`. This builds the backend dependencies *and* the web app. Express then serves `client/dist` from the same origin.
- **Start:** `npm start`, which runs `node src/server.js`.

- **Health check:** `GET /api/health` returns `{ "status": "ok", "time": ... }`.
- **Generated secrets:** `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` and `CHAT_ENCRYPTION_KEY` use `generateValue: true`, so Render creates them. You never see or set them.
- **Dashboard-only values** (`sync: false`): `MONGO_URI`, the `SMS_*` credentials, the `CLOUDINARY_*` values, `GROQ_API_KEY`, `FCM_SERVICE_ACCOUNT` and `TRACKER_WEBHOOK_SECRET`.
- **Fixed values:** `NODE_ENV=production`, `APP_TIMEZONE=Asia/Kolkata`, `TRUST_PROXY=1`, `CLIENT_URL` / `APP_URL` = `https://jnn.onrender.com`, `OTP_SMS_ENABLED=true`, `SMS_ALLOW_INSECURE_HTTP=true`, `DEV_OTP_PORTAL=false`, `TRANSPORT_STUDENT_VISIBILITY=all`.

> **Warning**
> `SMS_ALLOW_INSECURE_HTTP=true` is set because the college SMS vendor has no HTTPS endpoint (this is explained in `render.yaml` and `server/src/config/env.js`). The SMS API key, parents' numbers and OTPs therefore cross the internet **unencrypted** between Render and the vendor. This is a known, accepted risk. Ask the vendor for an HTTPS endpoint and switch it off as soon as one exists.

Sleeping and waking (free plan)

The Render free plan sleeps after 15 minutes without traffic. VEXON handles this in two places:

1. `server/src/utils/keepAlive.js`: while running on Render, the server pings its own public URL every 14 minutes, but **only between 05:30 and 22:00 IST**.
2. `.github/workflows/keep-alive.yml`: GitHub Actions wakes the server at about 05:32 and 05:47 IST each day.

The result: the first request at night takes about 30 seconds while the instance wakes. Free plans also have monthly instance-hour limits. The workflow comment estimates about 510 of 750 hours.

Required environment variables (names only)

The full annotated list is `server/.env.example`. **Never** put values into documents, chats or commits.

Group	Variable names
Server	`PORT`, `NODE_ENV`, `APP_TIMEZONE`, `TRUST_PROXY`, `CLIENT_URL`, `APP_URL`
Database	`DB_TARGET`, `MONGO_URI`, `MONGO_URI_ATLAS`
Auth (required in production)	`JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, `JWT_ACCESS_EXPIRES`, `JWT_REFRESH_EXPIRES`, `CHAT_ENCRYPTION_KEY`
Uploads	`UPLOAD_DIR`, `MAX_UPLOAD_MB`, `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`
Email	`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`, `PASSWORD_RESET_MINUTES`
Push	`FCM_SERVICE_ACCOUNT`, `FCM_SERVICE_ACCOUNT_FILE`, `EXPO_ACCESS_TOKEN`
Reminders	`DIGEST_TIME`
Bus tracking	`MAP_STYLE_URL`, `MAP_ATTRIBUTION`, `MAP_CSP_ORIGINS`, `TRANSPORT_STUDENT_VISIBILITY`, `TRANSPORT_LOCATION_RETENTION_DAYS`, `TRANSPORT_LIVE_SECONDS`, `TRANSPORT_STALE_SECONDS`, `GPS_MAX_AGE_SECONDS`, `GPS_MAX_FUTURE_SECONDS`, `GPS_MAX_ACCURACY_M`, `GPS_MAX_SPEED_MPS`, `GPS_MIN_INTERVAL_MS`, `GPS_HISTORY_INTERVAL_MS`, `TRIP_IDLE_MINUTES`, `TRIP_MAX_HOURS`, `TRACKING_LOST_MINUTES`, `BUS_APPROACHING_MINUTES`, `TRACKER_WEBHOOK_SECRET`
Gate-pass OTP and SMS	`OTP_SMS_ENABLED`, `OTP_PROVIDER`, `OTP_EXPIRY_MINUTES`, `OTP_LENGTH`, `OTP_MAX_ATTEMPTS`, `OTP_RESEND_COOLDOWN_SECONDS`, `SMS_API_KEY`, `SMS_SENDER_ID`, `SMS_OTP_TEMPLATE_ID`, `SMS_EXIT_TEMPLATE_ID`, `SMS_API_URL`, `SMS_ALLOW_INSECURE_HTTP`, `DEV_OTP_PORTAL`

Group	Variable names
Return-to-campus check	`COLLEGE_LATITUDE`, `COLLEGE_LONGITUDE`, `LOCATION_GEOFENCE_RADIUS_METERS`, `LOCATION_MAX_ACCURACY_METERS`, `LOCATION_MAX_AGE_SECONDS`, `RETURN_CREDENTIAL_MINUTES`
AI assistant	`GROQ_API_KEY`, `GROQ_MODEL`, `AI_WEB_SEARCH_ENABLED`, `AI_MAX_MESSAGE_LENGTH`, `AI_MAX_OUTPUT_TOKENS`, `AI_REQUEST_TIMEOUT_MS`, `AI_MAX_RETRIES`, `AI_RATE_LIMIT_PER_MINUTE`, `AI_RATE_LIMIT_PER_HOUR`, `AI_MAX_CONVERSATION_MESSAGES`, `AI_HISTORY_MESSAGES`, `AI_MAX_CONCURRENT`, `AI_MAX_QUEUE`, `AI_RAG_CHUNKS`
Keep-alive	`RENDER_EXTERNAL_URL` (set by Render), `KEEP_ALIVE_URL`, `KEEP_ALIVE`
Android build	`vexon.apiUrl` / `VEXON_API_URL`, `vexon.easProjectId` / `EAS_PROJECT_ID`, `google-services.json` (file, not committed)

Build and start commands

```bash
# Single-server production build (what Render does)
npm install --prefix server
npm install --include=dev --prefix client && npm run build --prefix client
NODE_ENV=production npm start --prefix server      # Linux/macOS syntax; set the variable on the host
    instead

# Android release APK (from mobile-kotlin/)
gradlew.bat assembleRelease      # app/build/outputs/apk/release/ - currently signed with the DEBUG key
```

> **Warning**
> The Kotlin release build is signed with the **debug key** until a real `signingConfig` is added (`mobile-kotlin/app/build.gradle.kts`, `mobile-kotlin/README.md`). That is fine for sideloading on campus phones, but not acceptable for the Play Store. If you ever lose a real release key, users cannot receive updates, so back the key up offline.

HTTPS, proxies and secure networking

- **HTTPS is required in production.** The cookies are named `__Secure-cc_refresh` and `__Host-cc_csrf` there, and browsers accept those prefixes only over HTTPS (`server/src/utils/tokens.js`, `server/src/middleware/security.js`). On plain HTTP, sign-in would appear to work but refresh and CSRF would fail.
- **HSTS** (one year) is sent only in production (`app.js` helmet config).
- `TRUST_PROXY` must equal the real number of proxy hops: 1 behind Render, 2 behind Cloudflare plus Render (`trustProxySetting` in `app.js`). A wrong value either lets clients spoof their IP or makes every visitor share one IP, which breaks rate limits and lockouts.
- **CORS and origins**: `CLIENT_URL` (comma-separated) must list the real public URL(s). The CSRF/origin guard and Socket.IO both use it.
- **Atlas network access**: the README suggests allowing `0.0.0.0/0` because Render has no static IP. That means the database relies entirely on its credentials, so use a long, unique database password and a database user with access to that one database only. (The Atlas settings themselves are **unverified**.)

Process management, logs and health checks

Concern	On Render (today)	On a VPS (future plan)
Keep the process alive and restart it on crash	Render	systemd service (in the VPS guide)
Graceful shutdown	`server.js` handles SIGINT/SIGTERM and closes the HTTP server	Same
Logs	Render dashboard, Logs (stdout/stderr)	`journalctl -u <service>`
Health	`/api/health` (Render health check)	An external uptime monitor polling `/api/health`
Error tracking	**None configured.** No Sentry or similar package exists	Consider adding one, as a deliberate dependency decision
Audit and security logs	`Activity`, `SecurityLog` in MongoDB; Admin, then Security & Logs	Same

> **Note**
> `/api/health` proves only that Node is answering. It does **not** check MongoDB. An outage of the database alone would still report "ok". Treat a database check as a possible improvement, not current behaviour.

Backups and tested restoration

There is **no automated backup in the repository**. For production on Atlas:

1. Check which backup features your Atlas tier includes (free tiers are limited). This is **unverified** from the repository.
2. Independently, take a scheduled `mongodump` from a trusted machine to encrypted storage (Guide 04).
3. Back up uploaded files: Cloudinary assets, or the `StoredFile` collection, which a `mongodump` includes.
4. **Restore drill** at least monthly: restore into a scratch database, start a local server against it with `DB_TARGET` and the URI pointing there, and sign in with a test account.

Monitoring, incidents and rollback

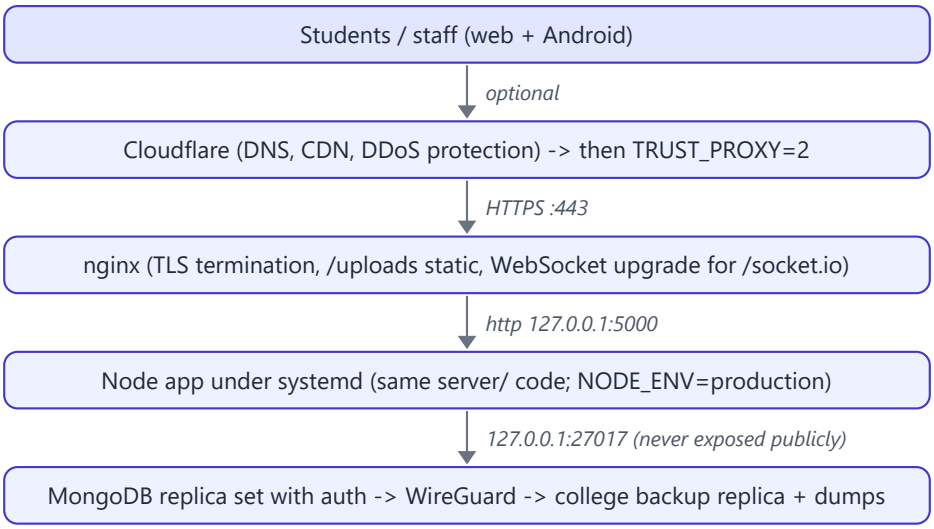
- **Watch:** the Render service status, `/api/health`, the Render logs for `[error]`, and Admin, then Security & Logs for spikes of `LOGIN_FAILED`.
- **Incident:** follow the checklist and report template in Guide 06.
- **Rollback (Render):** redeploy the last good commit from the Render dashboard, or `git revert <bad commit>` and push to the deploying branch. Data changes are **not** rolled back by a code rollback. That is why a backup comes before any migration.

Security and authorization checks before going live

- [ ] `NODE_ENV=production` (secrets are enforced; error messages masked; HSTS; secure cookies).
- [ ] Strong generated JWT and chat secrets (Render's `generateValue`). Changing `CHAT_ENCRYPTION_KEY` makes old chat messages unreadable.
- [ ] `CLIENT_URL` / `APP_URL` exactly match the public URL.
- [ ] `TRUST_PROXY` matches the proxy chain.
- [ ] `ALLOW_SELF_SIGNUP` is not `true` (accounts are created by admins).
- [ ] `DEV_OTP_PORTAL` off (it is always 404 in production anyway).
- [ ] The CI secret scan is green; `npm run verify:bundle` passes on the built client.
- [ ] The admin account password is strong and unique; demo or seed accounts do **not** exist in production.
- [ ] The role-permission tests pass (`npm test --prefix server`).

Planning a move to a VPS

> **Evidence status**
>
> **Planned, not implemented.** `docs/VEXON_VPS_Setup_Guide_Ubuntu24.pdf` (12 pages) describes a Hostinger KVM 4 VPS with Ubuntu 24.04: nginx with HTTPS, optional Cloudflare in front, a systemd service for the Node app, MongoDB 8.0 as a replica set with authentication, a WireGuard tunnel to a college machine holding a hidden replica, and off-site dumps. Its own header says the commands **were not run**. Treat it as a draft plan.



Migration steps, in order:

1. Build the VPS and harden it (SSH keys, firewall, updates) by following the VPS guide on a **fresh** server or snapshot.
2. Install the same Node major version used in CI (20) or newer; the engines field says `>=18.18`.
3. Restore a *copy* of production data (`mongorestore`) and run the app against it under a temporary hostname.
4. Run the full test suite on that machine; smoke-test every critical workflow; check that WebSockets work through nginx.
5. Point the Kotlin app at the new URL (rebuild with `-Pvexon.apiUrl=...`, or change it on the login screen) and test on real phones.

6. Plan the cutover window: a final dump from Atlas, restore, switch DNS, and keep Render on standby for rollback.
7. Watch closely for a week, and keep Atlas read-only for a while as a fallback.

Local build vs production-ready system

A working local build has...	A production-ready system also needs...
`npm run dev` working on your PC	HTTPS, correct `CLIENT_URL`, `TRUST_PROXY`, secure cookies
Development secrets (with warnings)	Strong secrets set only on the host
A seeded demo database	Real accounts created by admins; no demo users
Files on local disk	Durable file storage (Cloudinary or backed-up `StoredFile`)
Tests passing on your machine	CI green on the exact commit deployed
Logs in your terminal	Retained logs, an uptime check, someone who watches them
"It worked when I tried"	Backups **with tested restores**, a rollback plan, and an incident process
A debug-signed APK	A properly signed release, with the key stored safely

Source references

`render.yaml` · `server/.env.example` · `server/src/config/env.js` · `server/src/app.js` · `server/src/server.js` · `server/src/utils/keepAlive.js` · `server/src/utils/tokens.js` · `server/src/middleware/security.js` · `server/src/utils/storage.js` · `.github/workflows/ci.yml` · `.github/workflows/keep-alive.yml` · `.github/workflows/secret-scan.yml` · `.github/workflows/zap-baseline.yml` · `scripts/verify-bundle.mjs` · `mobile-kotlin/app/build.gradle.kts` · `mobile-kotlin/README.md` · `docs/VEXON_VPS_Setup_Guide_Ubuntu24.pdf` · `README.md`